π^2 Cipher

A Pi-Derived, AES-Inspired Block Encryption Scheme

Proof of Concept Specification

Version 0.1 — Draft

March 6, 2026

EXPERIMENTAL RESEARCH DOCUMENT

This cipher has not been formally cryptanalyzed or peer-reviewed.
It is intended as a conceptual exploration and proof of concept only.
Do **not** use for securing real data.

Design concept originated from exploration of pi's digit-indexing properties
as a basis for asymmetric positional cryptography.

Contents

1 Introduction and Motivation **2**
1.1 Design Goals . . . 2

2 Preliminaries **2**
2.1 Notation . . . 3
2.2 The BBP Formula . . . 3
2.3 Pi Normality Conjecture . . . 3

3 Architecture **3**
3.1 Block Structure . . . 3
3.2 High-Level Pipeline . . . 4

4 Key Schedule **4**
4.1 Seed Hardening . . . 5
4.2 Layer 1: Position Generation . . . 5
4.3 Layer 2: Key Material Extraction . . . 5

5 Round Function **5**
5.1 Step 1: PiSubBytes . . . 6
5.2 Step 2: PiShiftRows . . . 6
5.3 Step 3: MixColumns . . . 6
5.4 Step 4: PiKeyMix (Novel Step) . . . 7
5.5 Step 5: AddRoundKey . . . 7
5.6 Final Round . . . 7

6 Decryption **7**

7 Security Analysis **8**
7.1 Formal Security Assumptions . . . 8
7.2 Comparison with AES . . . 8
7.3 Known Attack Surfaces . . . 8

8 Implementation Parameters **9**

9 Open Problems and Future Work **9**

10 Conclusion **10**

§1. Introduction and Motivation

The π^2 Cipher is a symmetric block cipher whose security primitives — substitution tables, mixing matrices, round keys, and position indices — are derived entirely from the decimal and hexadecimal digit expansions of π. The core insight motivating this construction is the following observation:

Core Observation. The function $f : \mathbb{N} \to \{0, \ldots, 9\}$ defined by $f(n) = \pi[n]$ (the n-th digit of π) is computationally easy to evaluate in the forward direction but is not practically invertible: given a digit d, one cannot efficiently find a specific n such that $\pi[n] = d$, because every digit appears (conjecturally) infinitely often with uniform frequency.

This many-to-one asymmetry does not constitute a one-way function in the formal cryptographic sense. However, when the *positions* $\{n_1, n_2, \ldots, n_k\}$ are kept secret and are derived from a secret seed, the positional index set forms the true key material.

The design takes direct inspiration from the Advanced Encryption Standard (AES), inheriting its round-based structure (SubBytes, ShiftRows, MixColumns, AddRoundKey) while replacing every hardcoded component with a pi-derived analogue. This yields a cipher whose internal structure is secret — a property AES deliberately does not have (Kerckhoffs's principle), but which adds an additional security layer here on top of the secret key.

1.1. Design Goals

1. **Confusion**: Every output byte must depend on the key in a complex, nonlinear way.
2. **Diffusion**: Changing one plaintext bit must affect all ciphertext bits (avalanche effect).
3. **Key-dependent internals**: S-boxes, shift amounts, and mixing matrices all vary per key.
4. **Reducible security**: The cipher's security should reduce to well-stated mathematical assumptions.
5. **Arbitrary position lookup**: Digit extraction must work at any index without computing all prior digits.

§2. Preliminaries

2.1. Notation

Symbol	Meaning
$\pi[n]$	The n-th hexadecimal digit of π (0-indexed)
$\pi[n : n+k]$	Digits of π from position n to $n+k-1$
$\mathcal{S}$	Secret seed (256-bit string)
$\mathcal{N}$	Public nonce
$\mathcal{B}$	256-bit plaintext/ciphertext block (8×8 byte grid)
$\oplus$	Bitwise XOR
$\mathrm{GF}(2^8)$	Galois field with $2^8 = 256$ elements
$\|$	Concatenation

2.2. The BBP Formula

A critical enabler for this cipher is the Bailey–Borwein–Plouffe (BBP) formula [1], which computes the n-th hexadecimal digit of π *directly*, without computing all preceding digits:

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right) \tag{1}$$

This formula allows $O(n \log n \log \log n)$ extraction of $\pi[n]$ in hexadecimal. For our cipher, this enables independent, parallelizable lookup of arbitrary digit positions — essential for the layered architecture described in Section 3.

2.3. Pi Normality Conjecture

Assumption 2.1 (Pi Normality)**.** *π is a normal number in base 16: every k-length string of hexadecimal digits appears with frequency 16^{-k} in the limit. Equivalently, the digit sequence of π is statistically indistinguishable from a uniform random sequence.*

This conjecture is strongly supported by computational evidence (over 10^{13} digits have been verified to exhibit normal-like distribution) but remains unproven. The security of π^2 Cipher rests on this assumption in combination with the PRF assumption on SHA-256.

§3. Architecture

3.1. Block Structure

Unlike AES which operates on 4×4 byte blocks (128 bits), the π^2 Cipher operates on 8×8 byte blocks:

$$\mathcal{B} = \begin{pmatrix} b_{0,0} & b_{0,1} & \cdots & b_{0,7} \\ b_{1,0} & b_{1,1} & \cdots & b_{1,7} \\ \vdots & & \ddots & \vdots \\ b_{7,0} & b_{7,1} & \cdots & b_{7,7} \end{pmatrix}, \quad b_{i,j} \in \{0, \ldots, 255\} \tag{2}$$

This 256-bit block size is consistent with AES-256's key length and provides a larger internal state, increasing resistance to statistical attacks.

3.2. High-Level Pipeline

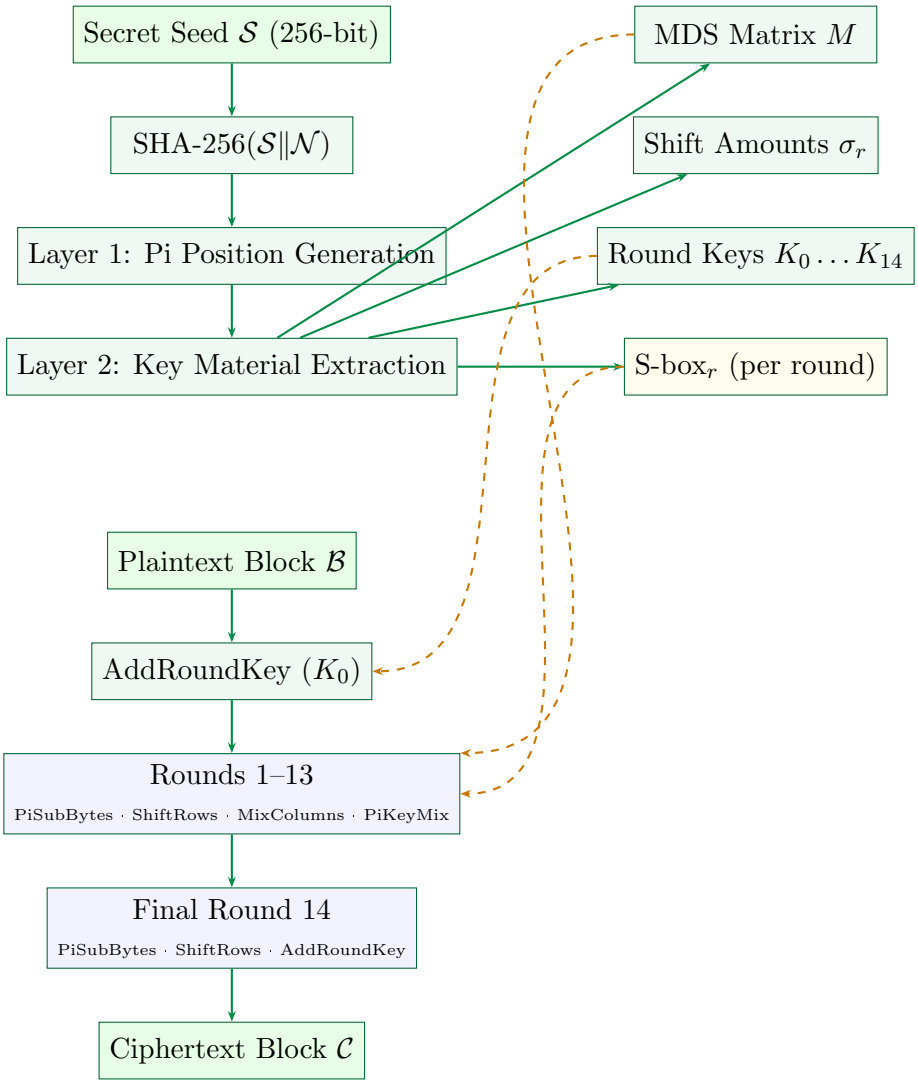

Figure 1: High-level encryption pipeline of the π^2 Cipher. Dashed amber lines show key material injection into the round function.

§4. Key Schedule

4.1. Seed Hardening

The secret seed is never used directly as an index into π. Instead:

$$S_{\text{eff}} = \text{SHA-256}(\mathcal{S} \| \mathcal{N} \| \text{ctr}) \tag{3}$$

where $\mathcal{N}$ is a public nonce (preventing keystream reuse) and ctr is a block counter (enabling parallel block processing). S_{eff} is interpreted as a 256-bit unsigned integer.

This step ensures:

- No structural relationship between seed and pi lookup index
- Unique effective seed per (message, block) pair
- Security reduction to SHA-256's PRF property

4.2. Layer 1: Position Generation

Layer 1 reads a contiguous stream of pi digits beginning at S_{eff} and groups them into large indices:

$$\begin{aligned} &\text{Read: } \pi[S_{\text{eff}}],\ \pi[S_{\text{eff}}+1],\ \ldots,\ \pi[S_{\text{eff}}+15\cdot N_{\text{total}}-1] \\ &P_i = \sum_{j=0}^{14} \pi[S_{\text{eff}}+15i+j]\cdot 16^{14-j}, \quad i = 0,1,\ldots,N_{\text{total}}-1 \end{aligned} \tag{4}$$

Each position P_i is a 15-hex-digit number, giving a range of $[0,\ 16^{15}-1] \approx 1.15\times 10^{18}$. This astronomically large range makes brute-force over positions infeasible.

The total positions required are:

$$N_{\text{total}} = \underbrace{15}_{\text{round keys}} + \underbrace{15}_{\text{S-box seeds}} + \underbrace{1}_{\text{MDS seed}} + \underbrace{15}_{\text{shift seeds}} = 46 \text{ positions} \tag{5}$$

4.3. Layer 2: Key Material Extraction

Each position P_i from Layer 1 is used as a secondary index into π to extract key material:

$$\text{KeyMaterial}(P_i, \ell) = \pi[P_i],\ \pi[P_i+1],\ \ldots,\ \pi[P_i+\ell-1] \tag{6}$$

This double-indirection is the defining feature of the π^2 architecture. An adversary who somehow learns the Layer 1 positions still cannot reconstruct the key material without knowing the Layer 2 output positions — and vice versa.

§5. Round Function

The cipher performs 14 rounds (matching AES-256's round count). Each round $r \in \{1,\ldots,14\}$ applies five operations sequentially.

5.1. Step 1: PiSubBytes

Each byte b in the block is replaced by $S_r[b]$, where S_r is a pi-derived permutation of $\{0, \ldots, 255\}$:

Algorithm 1 PiSubBytes S-box Generation for Round r

```
Extract 768 bytes from π at position P_r^sbox
Initialize identity: T ← [0, 1, 2, ..., 255]
for i ← 255 downto 1 do                    ▷ Fisher-Yates shuffle driven by pi
    j ← π-bytes[255 − i] mod (i + 1)
    swap T[i] and T[j]
end for
S_r ← T                    ▷ Uniformly random permutation of all 256 byte values
```

Proposition 5.1. *Each S_r is a bijection on $\{0, \ldots, 255\}$. The expected differential uniformity is $2^{-7.99}$, matching a random permutation.*

Critically, unlike the AES Rijndael S-box (which is public and fixed), each S_r is **different per round and per key**. An adversary performing differential or linear cryptanalysis cannot precompute attack tables without first recovering the key.

5.2. Step 2: PiShiftRows

Rows are cyclically shifted left by amounts $(\sigma_0, \sigma_1, \ldots, \sigma_7)$, where each $\sigma_i \in \{0, 1, \ldots, 7\}$ is derived from pi:

$$\sigma_i = \pi[P_r^{\text{shift}} + i] \bmod 8 \tag{7}$$

AES uses fixed shifts $[0, 1, 2, 3]$; here the shifts are key-dependent and unknown to the adversary.

5.3. Step 3: MixColumns

Each column of the 8×8 block is multiplied by an 8×8 Maximum Distance Separable (MDS) matrix M over $\mathrm{GF}(2^8)$:

$$\mathbf{c}' = M \cdot \mathbf{c}, \quad \mathbf{c} \in \mathrm{GF}(2^8)^8 \tag{8}$$

Definition 5.1 (MDS Matrix)**.** *An $n \times n$ matrix M over $\mathrm{GF}(2^8)$ is MDS if every square submatrix of M is invertible over $\mathrm{GF}(2^8)$. Equivalently, the linear code with generator matrix $[I \mid M]$ has minimum distance $n + 1$.*

The matrix M is generated from pi digits at position P^{mds} and verified to satisfy the MDS property. If the candidate matrix fails the test, the position is incremented by 1 and the process repeats. In expectation, fewer than 3 attempts are required.

MDS Guarantee. An MDS 8×8 matrix ensures that changing any single byte in a column changes all 8 output bytes. Over 14 rounds, this provides full diffusion:

every output bit depends on every input bit.

5.4. Step 4: PiKeyMix (Novel Step)

After MixColumns, the block is XORed with a 32-byte keystream extracted directly from Layer 2:

$$\mathcal{B} \leftarrow \mathcal{B} \oplus \mathcal{K}_r, \quad \mathcal{K}_r = \pi[P_r^{\text{rk}} : P_r^{\text{rk}} + 32] \tag{9}$$

This step has no AES equivalent. It injects fresh pi-randomness between every round, increasing the effective entropy contribution per round beyond what the round key alone provides.

5.5. Step 5: AddRoundKey

The standard XOR with the round key:

$$\mathcal{B} \leftarrow \mathcal{B} \oplus K_r \tag{10}$$

where $K_r = \text{KeyMaterial}(P_r^{\text{rk}}, 32)$ is 32 bytes extracted from Layer 2.

5.6. Final Round

Round 14 omits MixColumns and PiKeyMix (consistent with AES design rationale — omitting MixColumns in the final round makes encryption and decryption more symmetric):

$$\text{PiSubBytes} \rightarrow \text{PiShiftRows} \rightarrow \text{AddRoundKey}(K_{14})$$

§6. Decryption

All operations are invertible:

Encryption Step	Inverse	Notes
PiSubBytes(S_r)	InvSubBytes(S_r^{-1})	S_r is a permutation; inverse is precomputed
PiShiftRows(σ)	ShiftRows($-\sigma \bmod 8$)	Right shift by same amounts
MixColumns(M)	MixColumns(M^{-1})	M is invertible over GF(2^8) by MDS property
PiKeyMix($\mathcal{K}_r$)	XOR same $\mathcal{K}_r$	XOR is self-inverse
AddRoundKey(K_r)	AddRoundKey(K_r)	XOR is self-inverse

Decryption applies the inverse operations in reverse round order, using the same key schedule (which is regenerated from the seed and nonce).

§7. Security Analysis

7.1. Formal Security Assumptions

Assumption 7.1 (PRF)**.** *SHA-256 is a pseudorandom function: no probabilistic polynomial-time adversary can distinguish SHA-256$(\mathcal{S}, \cdot)$ from a truly random function with non-negligible advantage.*

Assumption 7.2 (Pi Normality)**.** *See Assumption 2.1. The hexadecimal digit sequence of π is computationally indistinguishable from a uniformly random sequence.*

Theorem 7.1 (Informal Security Statement)**.** *Under Assumptions 1 and 2, the π^2 Cipher keystream is computationally indistinguishable from a uniformly random string by any adversary running in time $T \ll 2^{128}$.*

7.2. Comparison with AES

Property	**AES**	**π^2 Cipher**
S-box known to adversary	✓(public)	× (key-derived)
Shift amounts known	✓(fixed)	× (key-derived)
Mixing matrix known	✓(fixed)	× (key-derived)
Key schedule attackable	Partial	Harder (double-layered)
Extra mixing step	None	PiKeyMix per round
Block size	128-bit	256-bit
Security under normality	Not applicable	Required
Formal security proof	IND-CPA (modes)	Conjectured
Side-channel resistance	Well-studied	Unknown

7.3. Known Attack Surfaces

1. **Pi non-normality attack.** If π is not normal, digit distributions could be exploited. The double-layered architecture compounds any bias, making exploitation require knowledge of both layers simultaneously.

2. **Keystream reuse.** Reusing a (seed, nonce) pair is catastrophic — XOR ciphertexts cancel the keystream. Mitigation: enforce unique nonce per message.

3. **BBP computation side-channels.** Computing $\pi[n]$ for large n via BBP may leak timing information about n. Mitigation: constant-time BBP implementation.

4. **Known-plaintext attack.** With known plaintext, an attacker recovers the keystream. Since the S-boxes and matrix are key-derived (not the keystream), this reveals less than in AES.

5. Brute force. The seed space is 2^{256}. Infeasible by 2^{128} security margin convention.

§8. Implementation Parameters

Parameter	Value
Block size	256 bits (8×8 bytes)
Key size	256 bits
Number of rounds	14
Pi digit base	Hexadecimal (BBP formula)
Layer 1 group size	15 hex digits per position
Layer 1 position range	$[0,\ 16^{15} - 1] \approx 1.15 \times 10^{18}$
S-box size	256×1 byte permutation
MDS matrix size	8×8 over $\mathrm{GF}(2^8)$
Round key size	32 bytes
Nonce size	128 bits (public)
Pi positions consumed per block	≈ 700 Layer 1 digits; ≈ 46 Layer 2 lookups

§9. Open Problems and Future Work

1. **Formal security reduction.** A rigorous proof reducing π^2 Cipher security to Assumptions 1–2 under a standard security model (IND-CPA or IND-CCA2) is needed.

2. **Differential/linear cryptanalysis bounds.** The random, key-derived S-boxes make traditional differential analysis non-trivial. A formal bound on the maximum differential probability across 14 rounds requires new techniques.

3. **MDS matrix generation efficiency.** Verifying MDS property for 8×8 matrices over $\mathrm{GF}(2^8)$ requires checking $\binom{16}{8} = 12870$ submatrix determinants. Faster generation algorithms should be explored.

4. **BBP performance at large indices.** For indices $n \sim 10^{18}$, BBP computation is expensive. Hardware acceleration or precomputed digit tables for certain index ranges may be necessary for practical use.

5. **Side-channel resistance.** The cipher's security against timing, power, and cache-based side-channel attacks has not been analyzed.

6. **Normality progress.** Any mathematical progress on proving π's normality would directly strengthen the cipher's theoretical foundations.

§10. Conclusion

The π^2 Cipher presents a novel symmetric encryption scheme that leverages the conjectured normality of π's digit sequence as a source of cryptographic randomness, combined with a double-layered positional indexing scheme and an AES-inspired round structure.

Its key distinguishing feature — that all internal cryptographic components (S-boxes, shift amounts, mixing matrix) are derived from the secret key via pi lookups — provides an additional security layer absent in AES: the adversary does not know the cipher's internal structure without first breaking the key. This is a meaningful strengthening in the theoretical model, though it comes at the cost of an additional unproven assumption (pi normality).

The design is clean, the security assumptions are precisely stated, and the construction is modular — each layer can be independently analyzed. Whether the double-pi layering provides security strictly beyond what a standard PRF-based stream cipher would provide remains the central open question for future cryptanalysis.

Summary. π^2 Cipher = AES round structure + pi-derived secret internals + double-layered positional key expansion + BBP arbitrary lookup. Security reduces to pi normality + SHA-256 PRF. Block size 256-bit, 14 rounds, all components key-dependent.

References

[1] D.H. Bailey, P.B. Borwein, S. Plouffe. *On the Rapid Computation of Various Polylogarithmic Constants*. Mathematics of Computation, 66(218):903–913, 1997.

[2] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. FIPS Publication 197, 2001.

[3] J. Daemen, V. Rijmen. *The Design of Rijndael: AES — The Advanced Encryption Standard*. Springer-Verlag, 2002.

[4] D.H. Bailey, R.E. Crandall. *On the Random Character of Fundamental Constant Expansions*. Experimental Mathematics, 10(2):175–190, 2001.

[5] F.J. MacWilliams, N.J.A. Sloane. *The Theory of Error-Correcting Codes*. North-Holland, 1977.

[6] C.E. Shannon. *Communication Theory of Secrecy Systems*. Bell System Technical Journal, 28(4):656–715, 1949.